

On the AP Exam, you will not be required to *write* a search algorithm; however, you will be required to *recognize* a search algorithm and trace it, which can be done the easy way if you understand the algorithm, or the hard way by actually tracing the code, step by step. Try to identify the key characteristics of each search; for example, the “sum divided by 2” line is a good indicator of a binary search. The enhanced-`for` loop will traverse the entire array, which may indicate a sequential search.

6 Mark for Review

Which of the following statements is true regarding search algorithms?

- ☐ (A) Searching for a Twix in a row of unsorted candy is most efficient using a binary search.
- ☐ (B) Searching for a Twix in a row of sorted candy is most efficient using a sequential search.
- ☐ (C) Searching for a Twix in a row of sorted candy is most efficient using a binary search.
- ☐ (D) None of these

Here's How to Crack It

Searching is usually more efficient using a binary search, provided the array is sorted—eliminate (A), since you can't use a binary search on unsorted data. However, a binary search will require more than one comparison if the target is not in the middle, whereas a linear search (also called a sequential search) will need only one comparison if the target is first in line. Therefore, each of the situations presented in (B) and (C) could be the most efficient, depending on the position of the Twix. The answer is (D).

7



Mark for Review

Assuming `target` is in array `a`, which of the following methods will correctly return the index of `target` in sorted array `a`?

- I.

```
public int findTarget(int[] a, int target)
{
    for (int x = 0; x < a.length; x++)
        if (a[x] == target)
            return x;
}
```
- II.

```
public int findTarget(int[] a, int target)
{
    int k = 0;
    for (int x : a)
    {
        if (x == target)
            return k;
        k++;
    }
    return -1;
}
```
- III.

```
public int findTarget(int[] a, int target)
{
    int f = 0, h = a.length, g = 0;
    for (int i = 0; i < h; i++)
    {
        g = (f + h)/2;
        if (a[g] < target)
            f = g + 1;
        else if (a[g] > target)
            h = g - 1;
    }
    if (a[g] == target)
        return g;
    return -1;
}
```

☐ (A) I only☐ (B) II only☐ (C) II and III only☐ (D) I, II, and III

Here's How to Crack It

Notice that Options I and II look like linear searches, while III looks like a binary search. Option I will not compile because a non-void method must return data. An `if` statement without an `else` implies data may not be returned—even though the assumption states that will not occur—so the compiler will stop here. Eliminate (A) and (D). Both (B) and (C) include Option II, so it must be correct; skip to Option III. Note that the variables are a bit confusing—but front/middle/back correspond to `f/g/h`, so they are in alphabetical order. Option III is a nice binary search, and it avoids the return problem in Option I by returning `-1` if target is not found, a common practice when writing search algorithms. Therefore, Options II and III are correct and (C) is the answer.

Read Carefully!

This question is about a sorted array! Be sure you read the entire question carefully before you dive in.

SORTS

Probably the most useful algorithms you will learn in AP Computer Science A are **sorting algorithms**—but they are also probably the most difficult to understand. If you've been keeping up within this book so far, then you have a good start.

As the name implies, sorting algorithms will take data in an array and rearrange it into a particular order. We already know that this technique is useful if we want to search for data using the binary search algorithm. But imagine this automated process in real life: a process that automatically sorts the cups in your cabinet in height order, a process that automatically takes your homework assignments and arranges them in time order, and a process that sorts your to-do list in priority order. Needless to say, sorting algorithms are extremely powerful.

The **selection sort** is the first sorting algorithm we will discuss, and one of the three sorting algorithms you need to know for the AP Exam. This is a search-and-swap algorithm, so you should remember that the selection sort searches and swaps. Similar to a sequential search, the sort will first traverse the array for the lowest value. Once it finds the lowest value, it will swap its position in the array with the data at index 0. Now the first element is sorted. The process then repeats for index 1. The rest of the array will be searched for the lowest value and swapped with the data at index 1. Note that if the lowest value is already in position, it will stay there.

Consider the array below. We would like to sort this array from least to greatest.

8	6	10	2	4
---	---	----	---	---

Our strategy will be to first find the smallest element in the array and put it in the first position. We will then find the smallest of the remaining elements and put that in the second position. We will continue to do this until the array is ordered.

We can start by looking at every element in the array (starting with the first element) and finding the smallest element. It's easy for a person to quickly glance through the array and see which element is smallest, but the sorting algorithm that we will implement can compare only two elements at once. So here's how we can find the smallest element: take the number in the first cell in the array and assign it to a variable called `smallestSoFar`. We'll also assign the position of that value to a variable called `position`. In this case, `smallestSoFar` will equal 8 and `position` will be 0. Note that even though we are assigning 8 to `smallestSoFar`, the first cell of the array will contain 8; we didn't actually remove it.

```
smallestSoFar = 8;
position = 0;
```

8	6	10	2	4
---	---	----	---	---

Next we'll walk through the array and compare the next value to `smallestSoFar`. The next value is 6, which is less than 8, so `smallestSoFar` becomes 6 and `position` becomes 1.

```
smallestSoFar = 6;
position = 1;
```

8	6	10	2	4
---	---	----	---	---

Now let's look at the next value in the array. 10 is larger than 6, so `smallestSoFar` remains 6.

```
smallestSoFar = 6;
position = 1;
```

8	6	10	2	4
---	---	----	---	---

The next value in the array is 2: 2 is smaller than 6.

```
smallestSoFar = 2;
position = 3;
```

8	6	10	2	4
---	---	----	---	---

And finally, we look at the last element, 4. Because 4 is greater than 2, and we are at the end of the array, we know that 2 is the smallest element.

```
smallestSoFar = 2;
position = 3;
```

8	6	10	2	4
---	---	----	---	---

Now we know that 2 is the smallest element in the array. Because we want to order the array from least to greatest, we need to put 2 in the first cell in the array. We don't simply want to overwrite the 8 that is in the first cell, though. What we'll do is swap the 2 with the 8 to get

2	6	10	8	4
---	---	----	---	---

We now have the smallest element in place. Next we'll need to find the second smallest element in the array. We can do this using the same approach we employed to find the smallest element. Because we know that 2 is the smallest element, we have to look at the elements only in positions 1 to 4 for the second smallest element.

Start by assigning 6 to `smallestSoFar` and 1 to `position` and then compare 6 to 10. Six is the smaller element. Next, compare 6 to 8; 6 is still the smaller element. Finally, compare 6 to 4; 4 is smaller, and because we have no more elements in the array, 4 must be the second smallest element in the array.

Swap 4 with the second element in the array to get

2	4	10	8	6
---	---	----	---	---

Make another pass through the array to find the third smallest element, and swap it into the third cell. The third smallest element is 6.

2	4	6	8	10
---	---	---	---	----

Finally, we look at the last two elements. Eight is smaller than 10, so we don't need to do anything. Our array is now sorted from least to greatest.

Implementation of a Selection Sort

Here is how a selection sort can be implemented in Java. The following implementation will sort the elements from least to greatest and will begin by sorting the smallest elements first.

```
//precondition: numbers is an array of ints
//postcondition: numbers is sorted in ascending order
1      public static void selectionSort1(int[] numbers)
2      {
3          for (int i = 0; i < numbers.length - 1; i++)
4          {
5              int position = i;
6              for (int k = i + 1; k < numbers.length; k++)
7              {
8                  if (numbers[k] < numbers[position])
9                  {
10                     position = k;
11                 }
12             }
13             int temp = numbers[i];
14             numbers[i] = numbers[position];
15             numbers[position] = temp;
16         }
17     }
```

How could this be useful? Consider a case in which you have an unsorted array of 1,000 `Student` objects, and each `Student` object has a method that returns a grade point average for that `Student`. What if you would like to find the five students with the highest grade point average? In this case, it would be a waste of

time to sort the entire array. Instead, we can just run through five cycles of the second implementation of the selection sort shown above, and the top five students will be sorted.

The **insertion sort** is a little less intuitive. Rather than traversing the entire array, it compares the first two elements and, depending on the comparison, inserts the second value “in front” of the first value into index 0, moving the first value to index 1. The first two elements are now sorted. Then the third element is checked, and the inserting continues. Note that here, also, an already sorted element will remain in its position.

Below is an array with 9 elements. This array is sorted from least to greatest except for the last element.

2	3	5	8	11	14	17	22	15
---	---	---	---	----	----	----	----	----

We would like to move 15 so that the entire array is in order. First, we’ll temporarily assign 15 to a variable. This will give us room to shift the other elements to the right if needed.

temp = 15

2	3	5	8	11	14	17	22	
---	---	---	---	----	----	----	----	--

We then compare 15 to the first element to its left: 22. Because 22 is larger than 15, we shift 22 to the right.

temp = 15

2	3	5	8	11	14	17	->	22
---	---	---	---	----	----	----	----	----

We then compare 15 to the next element: 17. Because 17 is larger, we shift that to the right also.

temp = 15

2	3	5	8	11	14	->	17	22
---	---	---	---	----	----	----	----	----

Next we compare 15 to 14. Because 15 is larger, we don’t want to shift 14 to the right. Instead, we insert 15 into the empty cell in the array. Now the array is correctly sorted.

Insert 15:

2	3	5	8	11	14	15	17	22
---	---	---	---	----	----	----	----	----

Now we’ll look at how we can use the idea illustrated above to sort an entire array. This example will start at the beginning of the sorting process.

Here is the array that we are going to sort.

8	6	7	10
---	---	---	----

First, we'll look at just the first two elements of the array to make sure that they are sorted relative to each other.

8	6	7	10
---	---	---	----

To do this, we'll pull 6 (the number that is farthest to the right in our subarray) out of the array and temporarily assign it to a variable. We'll then compare 6 to 8. Because 8 is larger, shift 8 to the right and then put 6 in the cell where 8 was.

temp = 6

8	->	7	10
---	----	---	----

Here's what the array looks like.

6	8	7	10
---	---	---	----

Now we need to put 7 in the proper place relative to 6 and 8. We start by assigning 7 temporarily to a variable.

temp = 7

6	8		10
---	---	--	----

We then compare 7 to the first number to its left: 8. Because 7 is less than 8, we shift 8 one place to the right.

temp = 7

6	->	8	10
---	----	---	----

Next, we'll compare 7 to the next number in the array: 6. Because 6 is less than 7, we don't want to shift 6 to the right. Instead, we will put 7 in the second cell. Our array now looks like the following:

6	7	8	10
---	---	---	----

Now we need to put 10 in its place relative to the first 3 elements in the array.

6	7	8	10
---	---	---	----

temp = 10

6	7	8	
---	---	---	--

**Bonus Tips and Tricks...**

Check us out on YouTube
for additional
test-taking tips and must-
know strategies at
[youtube.com/
ThePrincetonReview](https://www.youtube.com/ThePrincetonReview)



First we compare 10 to 8; because 8 is smaller than 10, we don't need to shift 8 to the right. In fact, we can put 10 right back into the cell from which it came.

6	7	8	10
---	---	---	----

Implementation of an Insertion Sort

Here is how an insertion sort can be implemented in Java.

```
//precondition: x is an array of integers; x.length >= 0
//postcondition: x is sorted from least to greatest
1  public static void insertionSort(int[] x)
2  {
3      for (int i = 1; i < x.length; i++)
4      {
5          int temp = x[i];
6          int j = i - 1;
7          while (j >= 0 && x[j] > temp)
8          {
9              x[j + 1] = x[j];
10             j--;
11         }
12         x[j + 1] = temp;
13     }
14 }
```

Note that like the selection sort, the insertion sort contains nested loops. In this case, we have a while loop nested within a for loop.

The for loop, beginning on line 3, proceeds from index 1 to the end of the array. The while loop goes through the array from i to 0 and shifts elements that are larger than $temp$ to the right on line 9. On line 12, we put the value of $temp$ into its proper place in the array.

Another type of sort used on the AP Exam is the **merge sort**. This is a more complex type of sort that uses **recursion**, which is the technique that uses a method to call itself. (Recursion will be further discussed in Chapter 11.) A merge sort is like a divide-and-conquer. An array is split into two pieces. Each piece is sorted. The two sorted pieces are then merged together into one sorted list. In order to sort each of the two pieces, the same divide-and-conquer method is used.

Below is an array with 8 elements:

14	8	9	3	5	4	21	12
----	---	---	---	---	---	----	----

The merge sort will divide this array into two pieces:

14	8	9	3
----	---	---	---

5	4	21	12
---	---	----	----

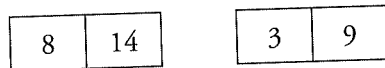
Let's look at just the left half of the array. This array is divided into two pieces:



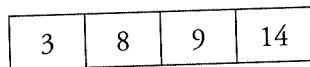
Each of these pieces is similarly divided into two pieces:



Since each remaining piece contains only one element, simply order the two from left to right:



Now merge the two sorted segments. Compare the first element in each piece. The smaller is 3, so this is the first element of the merged array. Now compare the first element in each piece that hasn't already been copied. These are 8 and 9. The smaller is 8, so this is the second element of the merged array. Again, compare the first element in each piece that hasn't already been copied. These are 14 and 9. The smaller is 9, so this is the third element of the merged array. Since the second array has no remaining elements that haven't already been copied, add the remaining element(s) from the first array into the merged array. The only remaining element is 14, so this is the fourth element:



Follow a similar process for the right half of the first array. Divide into two pieces:



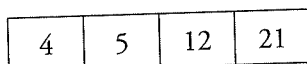
Each of these pieces is similarly divided into two pieces:



Since each remaining piece contains only one element, simply order the two from left to right:



Now merge the two sorted segments. Compare the first element in each piece. The smaller is 4, so this is the first element of the merged array. Now compare the first element in each piece that hasn't already been copied. These are 5 and 12. The smaller is 5, so this is the second element of the merged array. Since the first array has no remaining elements that haven't already been copied, add the remaining elements from the second array into the merged array. The remaining elements, 12 and 21, become the third and fourth elements of the merged array:



Thus, the two sorted pieces of the original array look like this:

3	8	9	14
---	---	---	----

4	5	12	21
---	---	----	----

Merge the two sorted arrays. Compare 3 and 4 to get that the first element is 3. Compare 8 and 4 to get that the second element is 4. Compare 8 and 5 to get that the third element is 5. Compare 8 and 12 to get that the fourth element is 8. Compare 9 and 12 to get that the fifth element is 9. Compare 14 and 12 to get that the sixth element is 12. Compare 14 and 21 to get that the seventh element is 14. Since the first array has no uncopied elements, 21, which is the only remaining uncopied element in the second array, becomes the eighth element of the merged array. Below is the final array:

3	4	5	8	9	12	14	21
---	---	---	---	---	----	----	----

Implementation of a Merge Sort

Here's how a merge sort can be implemented in Java.

```
//precondition: x is an array in integers; x.length >= 0
//postcondition: x is sorted from least to greatest
1 public static void mergeSort (int[] x)
2 {
3     int[] temp = new int[x.length];
4     mergeSortHelper(x, 0, x.length - 1, temp);
5 }
6 public static void mergeSortHelper (int[] x, int lowIndex,
7 int highIndex, int temp)
8 {
9     if (lowIndex < highIndex)
10    {
11        int mid = (lowIndex + highIndex) / 2;
12        mergeSortHelper(x, lowIndex, mid, temp);
13        mergeSortHelper(x, mid + 1, highIndex, temp);
14        merge(x, lowIndex, mid, highIndex, temp);
15    }
16 public static void merge(int[] x, int lowIndex, int
17 mid, int highIndex, temp)
18 {
19     int l = lowIndex;
20     int m = mid + 1;
21     int n = highIndex;
22     while (l <= mid && m <= highIndex)
23     {
24         if (x[l] < x[m])
25         {
26             temp[n] = x[l];
27             l++;
28         }
29         else
30         {
31             temp[n] = x[m];
```

```

31         m++;
32     }
33     n++;
34 }
35 while (l <= mid)
36 {
37     temp[n] = x[l];
38     l++;
39     n++;
40 }
41 while (m <= highIndex)
42 {
43     temp[n] = x[m];
44     m++;
45     n++;
46 }
47 for (n = lowIndex; n <= highIndex; n++)
48 {
49     x[n] = temp[k];
50 }
51 }

```

This is clearly a more complex sort than the other two and it involves the use of multiple methods. The first method splits the array into two pieces, the second sorts the individual pieces, and the third merges the two pieces into one sorted array.

CHAPTER 8 KEY TERMS

array
 initializer list
 index numbers
 traverse
 full
`ArrayIndexOutOfBoundsException`
 enhanced-for loop
 null
`NullPointerException`
 sequential search
 binary search
 search algorithms
 sorting algorithms
 selection sort
 insertion sort
 merge sort
 recursion

CHAPTER 8 REVIEW DRILL

Answers to review questions can be found in Chapter 12.

1 Mark for Review

What is the length of the array `numbers` after the following code executes?

```
String word = "Bear";  
int[] numbers = new int[word.length() - 1];
```

(A) 1

(B) 3

(C) 4

(D) The array will not be created due to a compile-time error.

2 Mark for Review

Consider the following code segment:

```
String[] s = new String[2];  
String[] t = {"Michael", "Megan", "Chelsea"};  
s = t;  
System.out.print(s.length);
```

What is printed as a result of executing the code segment?

(A) 2

(B) 3

(C) Nothing will be printed due to a compile-time error.

(D) Nothing will be printed due to a run-time error.

3

 Mark for Review

Consider the following code segment:

```
final int[] a1 = {1, 2};  
int[] b1 = {3, 4};  
a1 = b1;  
System.out.print(a1[1]);
```

What is printed as a result of executing the code segment?

(A) 2

(B) 4

(C) Nothing will be printed due to a compile-time error.

(D) Nothing will be printed due to a run-time error.

4

 Mark for Review

Consider the following code segment:

```
final int[] myArray = {1, 2};  
myArray[1] = 3;  
System.out.print(myArray[1]);
```

What is printed as a result of executing the code segment?

(A) 2

(B) 3

(C) Nothing will be printed due to a run-time error.

(D) Nothing will be printed due to a compile-time error.

5



Mark for Review

Consider the following incomplete method:

```
public static int mod3(int[] numbers)
    int count = 0;
    for (int i = 0; i < numbers.length; i++)
    {
        /* mystery code */
    }
    return count;
}
```

Method `mod3` is intended to return the number of integers in the array `numbers` that are evenly divisible by 3. Which of the following code segments could be used to replace `/* mystery code */` so that `mod3` will work as intended?

- I.

```
if (i % 3 == 0)
{
    count++;
}
```
- II.

```
if (numbers[i] % 3 == 0)
{
    count++;
}
```
- III.

```
while (numbers[i] % 3 == 0)
{
    count++;
}
```

(A) II only

(B) III only

(C) I and II

(D) II and III

6 Mark for Review

Assume that an array `arr` of integer values has been declared and initialized with random numbers.

Which of the following code segments correctly swaps the values stored in `arr[0]` and `arr[3]`?

(A) `arr[0] = 3;`
`arr[3] = 0;`

(B) `arr[0] = arr[3];`
`arr[3] = arr[0];`

(C) `int k = arr[3];`
`arr[3] = arr[0];`
`arr[0] = k;`

(D) `int k = arr[3];`
`arr[0] = arr[3];`
`arr[3] = k;`

7 Mark for Review

Consider the following code segment:

```
int [] scores = {75, 63, 52, 80};

for (int s : scores)
{
    if (s < 75)
    {
        s += 5;
    }
}
for (int s : scores)
{
    System.out.print(s + " ");
}
```

What is printed as a result of executing the code segment?

(A) 75 63 52 80

(B) 75 68 57 80

(C) 80 68 57 80

(D) Nothing will be printed due to a compile-time error.

8**Mark for Review**

Consider the following code segment:

```
String [] names = {"Abi", "Brianna", "Jada", "Kiara", "Tiffany", "Vanessa"};
int middle = names.length/2 - 1;

for (int x = 0; x < middle; x++)
{
    String hold = names[x];
    names[x] = names [names.length - x - 1];
    names[names.length - x - 1] = hold;
}

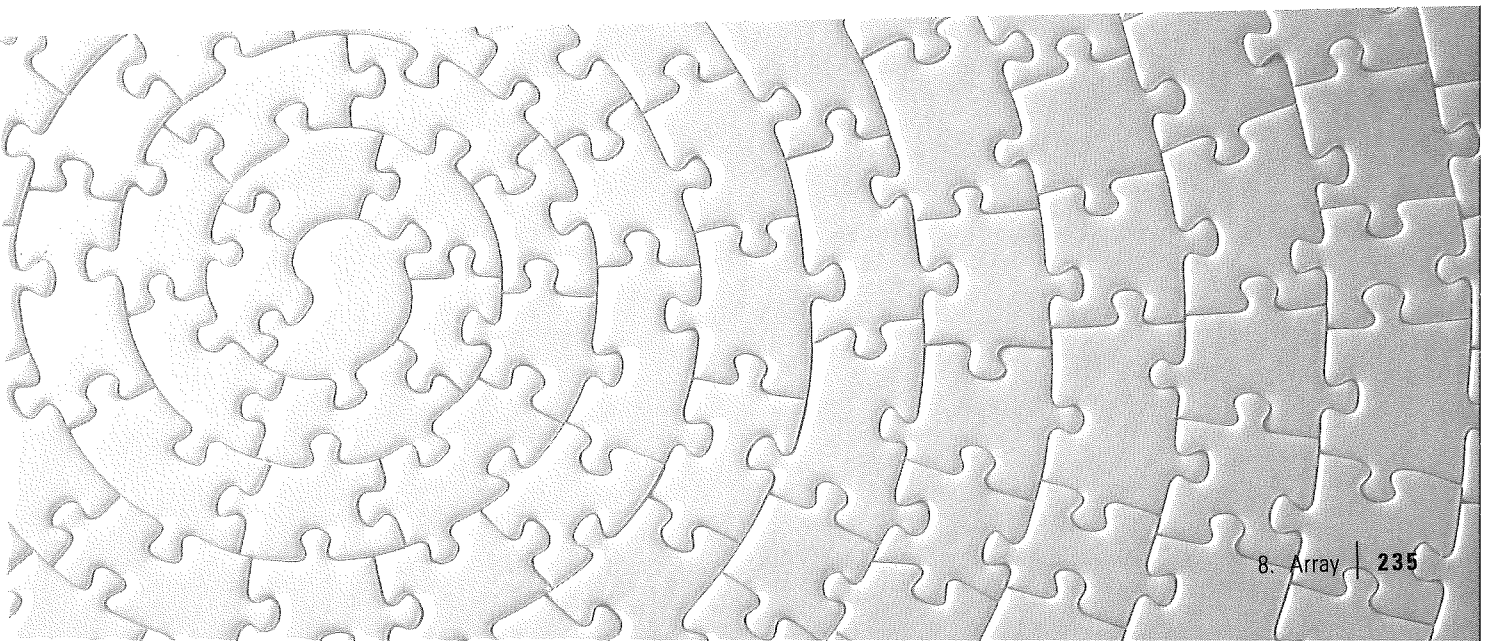
for (String n: names)
{
    System.out.print(n + " ");
}
```

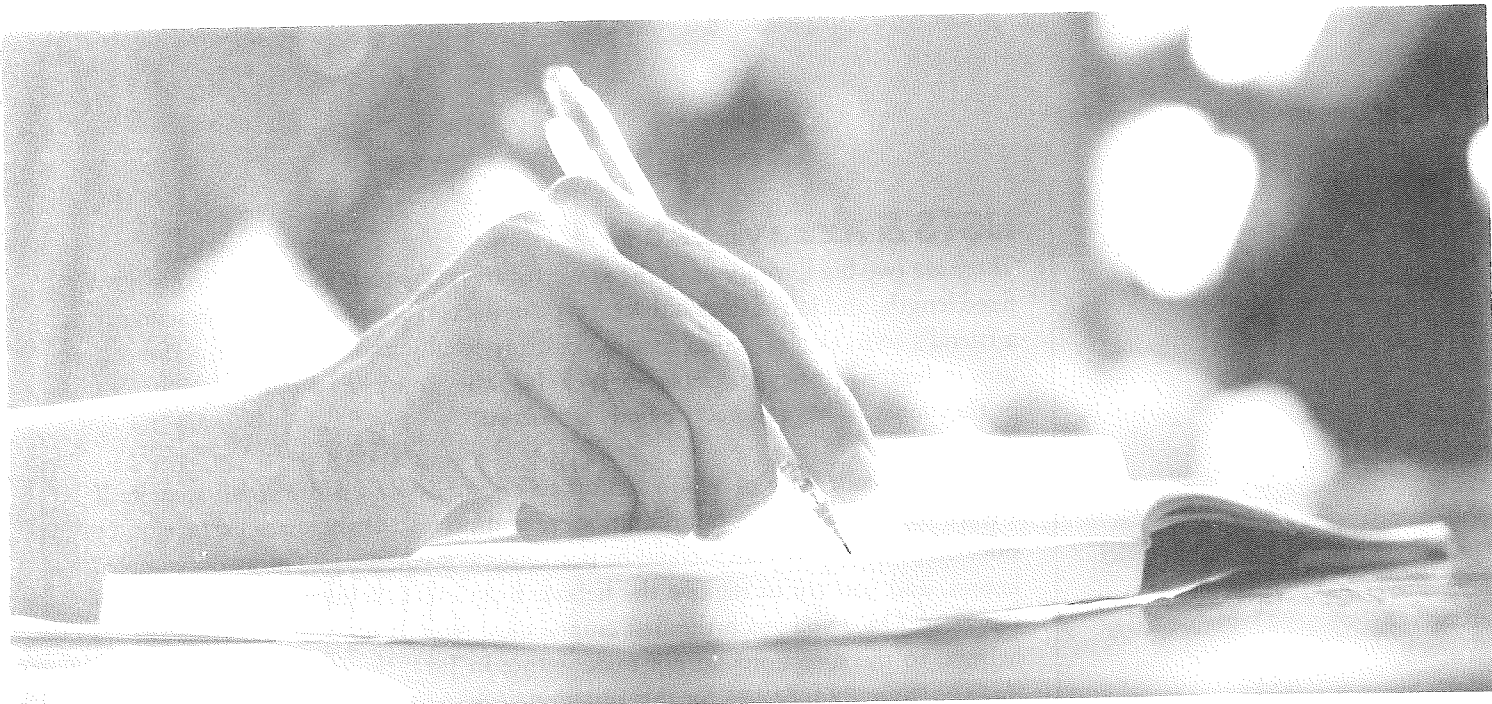
What is printed as a result of executing the code segment?

- ☐ (A) Abi Brianna Jada Kiara Tiffany Vanessa
- ☐ (B) Vanessa Tiffany Kiara Jada Brianna Abi
- ☐ (C) Vanessa Tiffany Jada Kiara Brianna Abi
- ☐ (D) Vanessa Brianna Jada Kiara Tiffany Abi

Chapter 8 Summary

- An array is a grouping of a fixed length of variables of the same data type or objects of the same class.
- Arrays are indexed beginning at 0 and ending at length -1 .
- An enhanced-`for` loop automatically performs the loop for each element of an array.
- A sequential search looks for an element of an array beginning at index 0 until the desired element is found and returns the index. If the element is not found, it returns -1 .
- A binary search begins searching in the middle of an array and moves to higher or lower indices to find the desired element. A binary search tends to be more efficient but can be used only on an ordered list.
- A selection sort orders an array by swapping the smallest elements to the first index and then repeatedly finding the second smallest element to swap into the second index, and so on.
- An insertion sort orders an array by finding the appropriate location for an element, shifting other elements to take its place, and then placing the element into the appropriate location.
- A merge sort orders an array by recursively dividing the array into smaller pieces. The smaller pieces are sorted and merged with other pieces.





Chapter 9

ArrayList

LISTS & ArrayLists

There are two big limitations of the powerful array structure in Java: an array has a fixed length, and it can store only one type of data. If you wanted to represent, say, a friend's collection of action figures, an array would require all of the action figures to be the same type. Generally speaking, they must *all* be flying superheroes, or they must *all* have protective body armor, etc.

Likewise, a collection of action figures represented as an array could store only a fixed number of action figures, no more and no less. If there are extra, unused slots, they stay there, which could be problematic if you have an array of helmeted heroes, and you try to remove the helmet from every element in every index of the array. Once you reach the first empty spot, there is no hero and therefore no helmet, so the directions do not make sense; in Java, the compiler will return a `NullPointerException` for this situation.

There are advantages and disadvantages to every structure that we study in Java.

An `ArrayList` addresses both of these issues. An **`ArrayList` object** is **dynamically sized**, expanding and compressing as elements are added and removed. An `ArrayList` can also store multiple types of data, without limit.

Let's use an example to understand the advantages and disadvantages of arrays versus `ArrayLists`. Consider your lunch bag that you take to school. If you wanted to represent your lunch bag using an array or `ArrayList` structure, which would be more accurate?

Naturally, the answer to this question depends on (1) the details of the objects in the bag—in this case, the types of lunch items—and (2) the programmer's choice of which is more appropriate. If your lunch contains a sandwich object, a fruit object, and a drink object, the `ArrayList` structure might be a better choice. Furthermore, as the components of the lunch are removed, the lunch bag theoretically shrinks (or can shrink). An `ArrayList` seems appropriate.

Let's consider the same lunch example, but this time, suppose the items are stored in a plastic container with compartments. Regardless of whether you have not yet eaten your lunch, you are done with your lunch, or it's currently anytime in between, the number and setup of the compartments do not change. We will discuss a workaround for "tricking" the array into thinking the lunch objects are all the same type. These facts and abilities render an array structure more appropriate than an `ArrayList`.

To further demonstrate the usefulness of an `ArrayList` structure, note that it is also possible to create a **typed `ArrayList`**, which allows only objects of the same type to be stored in the list. This structure combines useful aspects of both arrays and `ArrayLists`.

In order to instantiate an `ArrayList` called `lunchBag` that will store the various components of our lunch, we use the following line of code:

```
ArrayList lunchBag = new ArrayList();
```

Note that, unlike the syntax we use for instantiating an array, neither the type of object nor the length of the list is defined initially.

In order to access data from within the list, particular methods must be invoked; unlike for array structures in Java, there is not a convenient bracket notation available with lists. To return the second object, an `Apple` object, in the `ArrayList` and store it using the variable `food`, the line of code would be

```
Apple red = lunchBag.get(1);
```

There are several other useful methods available in the `List` class, and they are all mentioned in the AP Exam Quick Reference, although their functionality is (obviously) not given. These methods include `add`, `set`, `remove`, and `size`.

Bracket notation can be used only with array objects; lists must use the appropriate methods.

If the programmer decides it is more appropriate to keep the dynamic sizing ability of the `ArrayList` while fixing the type of data it can hold (as in an array), it would be instantiated as follows:

```
ArrayList<Apple> lunchBag = new ArrayList<Apple>();
```

One of the drawbacks of using `ArrayList` is that only objects can be stored in an `ArrayList`. The primitive data types `int` and `double` cannot be stored in `ArrayLists`. If programmers want to store integer and double data types, they must use the `Integer` or `Double` wrapper classes. `Integer` and `Double` objects can be created using integers and doubles, respectively, as parameters. For example,

```
Integer n = new Integer(5);
Double x = new Double(6.1);
```

To call the values, use the `intValue()` and `doubleValue()` methods. The following commands

```
int a = n.intValue();
double y = x.doubleValue();
```

assign `a = 5` and `y = 6.1`.

Additionally, the AP Computer Science Java Subset includes the static variables `MIN_VALUE` and `MAX_VALUE` of the `Integer` class. These store the minimum and maximum possible values of the integer data type.

Array	ArrayList
After an array is created, it cannot be resized.	ArrayLists will automatically resize as new elements are added.
No import statement is needed to use an array, unless the array holds elements that require an import statement.	You must import <code>java.util.ArrayList</code> , or use the full package name whenever you use an ArrayList.
Elements are accessed using index notation (e.g., <code>myArray[2]</code>).	Elements are accessed using methods of the ArrayList class (e.g., <code>myList.get(2)</code> , <code>myList.add("George")</code>).
Arrays can be constructed to hold either primitives or object references.	ArrayList instances can hold only object references, not primitives. The Integer and Double wrapper classes must be used to store integer and double primitives in an ArrayList.
Each array can be declared for only one type of element. For example, if an array is declared to hold strings, you cannot store an integer in it.	An ArrayList can hold a heterogeneous collection of objects. For example, the following is perfectly legal (though not recommended): <pre>ArrayList list = new ArrayList(); list.add(new String("A String")); list.add(new Integer(4));</pre>

INTRODUCTION TO DATA SETS

A **data set** is an organized collection of information relating to a particular topic or subject.

Some examples of data sets are

- Population data by state
- Exam grades for students in a class
- Statistics for players on a sports team
- Employee data for a company

The following table shows 2010 and 2020 census data for the ten most populous U.S. states:¹

Ten Most Populous U.S. States		
State	2010 Population	2020 Population
California	37,253,956	39,368,078
Texas	25,145,561	29,360,759
New York	19,378,102	19,336,776
Florida	18,801,310	21,733,312
Illinois	12,830,632	12,587,530
Pennsylvania	12,702,379	12,783,254
Ohio	11,536,504	11,693,217
Michigan	9,883,640	9,966,555
Georgia	9,687,653	10,710,017
North Carolina	9,535,483	10,600,823

How might a program represent the information in this data set? How can the program extract information and also generate new information (e.g., population change and percent of change)?

The information for each state can be represented by the following class:

```
public class State
{
    private String stateName;
    private int pop2010, pop2020, popChange;
    private double percentChange;

    public State(String name, int p2010, int p2020)
    {
        stateName = name;
        pop2010 = p2010;
        pop2020 = p2020;
        calculatePopChange();
        calculatePercentChange();
    }
    private void calculatePopChange()
    {
        //Calculates and saves the change in population
        //from 2010 to 2020
        popChange = pop2020 - pop2010;
    }
    private void calculatePercentChange()
    {
        //Calculates and saves the percent change in
        //population from 2010 to 2020
        //missing code
        //to be completed in the Review Drill
    }
}
```

¹ Source: [census.gov/topics/population.html](https://www.census.gov/topics/population.html)

The constructor takes three parameters: the name of the state, the population in 2010, and the population in 2020.

The method `calculatePopChange()` calculates and saves the State's population change from 2010 to 2020. The completion of the method `calculatePercentChange()`, which will compute and save the percent change from 2010 to 2020, is left as a problem in the Review Drill.

The entire data set (all ten States) can be stored in a one-dimensional array:

```
State[] top10=new State[10];
```

The array can be populated using data read from an input file. Using an input file will be covered in a later topic.

A program can analyze the entire data set in order to answer questions such as which state had the greatest percent change in population and which states had a decrease in population. The data set can also be sorted in a variety of ways, e.g., alphabetically or in decreasing order by percent change.

Another example of a data set is a collection of exam grades for students in a class. The information for each student can be represented by the following class:

```
public class Student
{
    private String last, first, letterGrade;
    private double average;
    private int[] grades;
    public Student(String last, String first, int[]grades)
    {
        this.last = last;
        this.first = first;
        this.grades = grades;
        calculateAverage();
        letterGrade = determineLetterGrade();
    }
    private void calculateAverage()
    {
        int len = grades.length;
        int total = 0;
        for (int g : grades)
            total += g;
        average = (double)total / len;
    }
    private String determineLetterGrade()
    {
        if (average >= 90)
            return "A";
        if (average >= 80)
            return "B";
        if (average >= 70)
            return "C";
        if (average >= 60)
            return "D";
        return "F";
    }
}
```

```

//accessors
public double getAverage()
{
    return average;
}
public String getLetterGrade()
{
    return letterGrade;
}
}

```

The constructor takes three parameters: the student's last and first names and an array of their exam grades.

The method `calculateAverage()` calculates the average of the Student's exam grades. The method `determineLetterGrade()` decides the letter grade for the Student based on their average.

The following class represents a data set for *all* the Students in a particular class:

```

public class TheClass
{
    private Student [] allStudents;
    private double classAverage;
    private Student studentWithHighestAverage;
    public TheClass(Student[] allStudents)
    {
        this.allStudents=allStudents;
        calculateClassAverage();
        studentWithHighestAverage();
    }
    private void calculateClassAverage()
    {
        //Calculates and saves the average grade for the
        //entire data set of Students
        double total = 0.0;
        for (Student s:allStudents)
            total += s.getAverage();
        classAverage = total / allStudents.length;
    }
    private void studentWithHighestAverage()
    {
        //Finds and saves the Student who has the highest
        //average
        Student highest = allStudents[0];
        for (int i = 1; i < allStudents.length; i++)
            if (allStudents[i].getAverage() > highest.getAverage())
                highest = allStudents[i];
        studentWithHighestAverage = highest;
    }
    public double getClassAverage()
    {
        return classAverage;
    }
    public Student getStudentWithHighestAverage()
    {
        return studentWithHighestAverage;
    }
}

```

The constructor takes one parameter, an array of `Student` objects. The method `calculateClassAverage()` computes and saves the average grade for all Students in the data set. The method `studentWithHighestAverage()` finds and saves the Student with the highest average in the data set.

The data set can also be sorted in various ways, e.g., alphabetically by last and first names; in decreasing order by average; in increasing order by letter grade.

USING TEXT FILES

Input text files are used by programs that need external data to perform their functions. For example, a grade calculation program might read in a text file that contains the name and exam grades for each student in a class. The program can save the information in an appropriate data structure, such as an array or `ArrayList`, and then retrieve and manipulate the data (e.g., to calculate the average for each student and for the entire class) as needed.

The data in a file is said to **persist**, which means that it exists even when the program is not executing.

The Java File Class

The Java File class contains methods that a program can use to work with external files.

A program using files must contain the following import statement:

```
import java.io.*;
```

The first step in using a file is to construct a `File` object as follows:

```
File fn = new File(pathname);
```

where `fn` is the name of the `File` object and `pathname` is a `String` that contains the path to the file.

For example, a file containing data for the ten most populous states in the U.S. might be located in the folder `C:/Java Programs` with the name `top10.txt`. The following line constructs a `File` object named `infile` for this:

```
File infile = new File("C:/Java Programs/top10.txt");
```

It is important to keep in mind that when a program uses a `File`, the method that uses the file *must* include `throws IOException` in its header. For example, if the instruction above is in the main method:

```
public static void main(String[] args) throws IOException
```

This allows the program to handle the situation where the file cannot be opened, e.g., if the path is not correct. An incorrect path results in a `FileNotFoundException`.

The Java Scanner Class

The Java Scanner class contains methods that allow a program to get (read) data from an input.

A program that uses the Scanner class must contain:

```
import java.util.Scanner; or
import java.util.*;
```

The first step is to construct a Scanner object. For the File object above:

```
Scanner scan = new Scanner(infile);
```

A program can use a while loop and the Scanner method `hasNext` to read data from an input file. For the example above:

```
while (scan.hasNext())
{
    //code to read and process data from the input file
}
```

`hasNext` returns true if there is still data to be read from the file, false if not.

The Scanner class provides methods to read different types of data from a file. The methods that are in the AP Computer Science A subset are:

<code>int nextInt()</code>	Returns the next int read from the file or input source. If the next int does not exist or is out of range, it will result in an <code>InputMismatchException</code>
<code>double nextDouble()</code>	Returns the next double read from the file or input source. If the next double does not exist, it will result in an <code>InputMismatchException</code> .
<code>boolean nextBoolean()</code>	Returns the next boolean read from the file or input source. If the next boolean does not exist, it will result in an <code>InputMismatchException</code> .
<code>String next()</code>	Returns the next String read from the file or input source

Let's look at a file named `top10.txt`, containing data for the ten most populous states in the U.S.

```
California 37253956 39368078
Texas 25145561 29360759
New_York 19378102 19336776
Florida 18801310 21733312
Illinois 12830632 12587530
Pennsylvania 12702379 12783254
Ohio 11536504 11693217
Michigan 9883640 9966555
Georgia 9687653 10710017
North_Carolina 9535483 10600823
```

The file consists of ten lines, one for each state. Each line contains three pieces of data: the state name (`String`), the 2010 population (`int`) and the 2020 population (`int`). The pieces of data are separated by **white space** (e.g., space or tab). This allows the appropriate Scanner methods to access them individually.

The `State` class was defined in the section on data sets.

```
public class State
{
    private String stateName;
    private int pop2010, pop2020, popChange;
    private double percentChange;
    public State(String name, int p2010, int p2020)
    {
        stateName = name;
        pop2010 = p2010;
        pop2020 = p2020;
        calculatePopChange();
        calculatePercentChange();
    }
    //other methods not shown
}
```

The data set for the ten states can be represented as an array of ten `State` objects.

```
State[] top10Pop=new State[10];
```

The following code will

- Read the name of the state (`String`) using the Scanner `next()` method
- Read the 2020 population (`int`) using the Scanner `nextInt()` method
- Read the 2010 population (`int`) using the Scanner `nextInt()` method
- Create a new `State` object as an element of `top10Pop`

for each line in top10.txt:

```
int i = 0;
while (scan.hasNext())
{
    String sName = scan.next();
    int p2010 = scan.nextInt();
    int p2020 = scan.nextInt();
    top10Pop[i] = new State(sName, p2010, p2020);
    i++;
}
```

The Scanner class also provides a method to read each line of a text file one at a time. This method is also in the AP Computer Science A subset:

String nextLine()	Returns the next line of text as a String read from the file or input source; can return the empty string if called immediately after another Scanner method that is reading from the file or input source
-------------------	--

The line can be split into separate pieces of data using the String split() method, which is also in the AP Computer Science A Java subset:

String [] split(String del)	Returns a String array where each element is a substring of this String, which has been split around matches of the given expression del.
-----------------------------	---

The following is an example of the use of nextLine() and split(String del):

The file numbers.txt contains data that consists of last and first names and phone numbers. Here are a few lines in numbers.txt:

```
Brown,John,(358)441-2435
Green,Vanessa,(358)222-9658
Gray,Matthew,(358)441-7654
Blue,Alison,(358)222-2468
```

The pieces of data are separated by “,” which serves as the **delimiter**. The delimiter is a String of one or more characters that do not appear in the data itself. When “,” is used the data is referred to as **comma-separated values**, or **CSV**.

The following is a class describing a person's first and last names and phone number:

```
public class PhoneNumber
{
    private String last, first, number;
    public PhoneNumber(String l, String f, String n)
    {
        last = l;
        first = f;
        number = n;
    }
}
```

The following code uses the `Scanner` `nextLine()` and `String` `split(del)` methods to get the data from `numbers.txt` and construct a `PhoneNumber` object:

```
File f = new File("numbers.txt");
Scanner scan = new Scanner(f);
String last, first, pNumber;
String line;
String[] temp = new String[3];
while (scan.hasNext())
{
    line = scan.nextLine();
    //The last name, first name and phone number are
    //separated by the delimiter ","
    //Split the line into the three pieces of data
    //saving each piece in the String array temp
    temp = line.split(",");
    last = temp[0]; //last name
    first = temp[1]; //first name
    pNumber = temp[2]; //phone number
    PhoneNumber pn = new PhoneNumber(last, first, pNumber);
    //code to save pn in a data structure not shown
}
```

Let's look at a line in `numbers.txt`:

Gray,Matthew,(358)441-7654

After the code

```
temp = line.split(",");
```

The `String` array `temp` is

Gray	Matthew	(358)441-7654
------	---------	---------------

`temp[0]` contains the last name

`temp[1]` contains the first name

`temp[2]` contains the phone number

There is one last `Scanner` method that is part of the AP Computer Science A Java subset:

<code>void close()</code>	Closes the scanner
---------------------------	--------------------

In the `PhoneNumber` example

```
scan.close();
```

can be called after the `while` loop.

CHAPTER 9 KEY TERMS

ArrayList object
dynamically sized
typed ArrayList
data set
persist
white space
delimiter
comma-separated values (CSV)